jackett-cookie-login-hardening

jackett_cookie_login_hardening_challenge.sh

Revision: 1 **Last modified:** 2026-08-15T12:00:00Z **Status:** active **Maintainer:** Track 11 — BOB-067 Lava-P4 port **Scope:** anti-bluff regression guard for the Jackett cookie-login hardening

Overview

Autonomous CI-runnable proof that `qBitTorrent-go/internal/jackett/client.go` correctly handles the Jackett dashboard cookie-session login the MANAGEMENT API requires. Ported from Lava `docs/autonomous-qa/PORTING-PLAYBOOK.md` §8 (P4) — the source-of-truth Go reference is `lava/lava-api-go/internal/jackett/client.go`. Boba's implementation already carries the same discipline (cookie jar, `CheckRedirect: ErrUseLastResponse`, `login()`, `doManaged()` login-on-302-retry, wrong-password safety net); this challenge is the runtime-signature ratchet that keeps it that way.

Root cause (Lava P4)

Jackett's MANAGEMENT API (`GET /api/v2.0/indexers`, per-indexer `/config`) authenticates via a DASHBOARD SESSION COOKIE — not via the apikey. An apikey-only management request is answered with **HTTP 302 → /UI/Login**. The apikey ONLY authorizes the Torznab `/results` + `/caps` feeds. A client that follows redirects lands on the HTML login page (200) and fails to decode it as JSON, masking the real cause; a client that does not follow gets a bare 302 and reports

“unreachable” / “http_302”. Either way the indexer catalog and per-indexer config path silently break — the exact bug the Lava porting document names.

Fix (Boba)

`qBitTorrent-go/internal/jackett/client.go`:

- `cookiejar.New(nil)` attached to the `*http.Client`.
- `CheckRedirect` returns `http.ErrUseLastResponse` (no auto-follow).
- `login()` POSTs `password=<admin>` to `/UI/Dashboard`; the `Set-Cookie` is captured into the jar. `net/http` stores response cookies BEFORE applying `CheckRedirect`, so this works whether the login answer is 200 or 302.
- Wrong-password safety net: 401/403 → explicit error; 200 with NO cookie → also error (“real Jackett re-renders the login page with 200 and no Set-Cookie on wrong password”).
- `doManaged()` wraps every management call: first response 302 → run `login()` → retry once with the (now-cached) session cookie.
- `NewClientWithPassword()` injects the admin password at runtime (§6.R / §11.4.10 no-literal-credentials).

Prerequisites

- `go` toolchain reachable on `PATH` (challenge SKIPs with reason if absent).
- `qBitTorrent-go/` module tree present.
- Live Jackett at `http://localhost:9117` is OPTIONAL — only the sibling `helixqa_jackett_fake_behavioral_equivalence_challenge.sh` uses it.

Usage

```
# GREEN — shipped regression guard (default is RED_MODE=0 in this
        script)
RED_MODE=0 bash challenges/scripts/
        jackett_cookie_login_hardening_challenge.sh

# RED — reproduces the pre-fix state (PASS if hardening ABSENT /
        broken)
RED_MODE=1 bash challenges/scripts/
        jackett_cookie_login_hardening_challenge.sh
```

What it proves

Runs three Go tests end-to-end via `go test -v -count=1` and asserts all three report `--- PASS:` (not just the top-level "ok" line, which a too-narrow `-run` filter would also print with zero tests):

1. `TestFakeJackettRefusesManagementWithoutCookie` — the fake refuses apikey-only management with 302 → `/UI/Login`. If this fails, every test built on the fake is worthless (§11.4.107(10) golden-bad).
2. `TestManagementCookieLogin_DiscoveryViaCookiePath` — an apikey-only `GetCatalog()` transparently acquires the session cookie and returns the discovery JSON (drives `doManaged`'s 302→login→retry path).
3. `TestManagementCookieLogin_ConfigurableAdminPassword` — a client built with the wrong admin password FAILs management instead of silently reporting success (verifies the no-cookie safety net).

Falsifiability rehearsal (paired §1.1 mutation)

Verified 2026-08-15 by patching `doManaged` to short-circuit past the 302→login retry:

```
// MUTATION BOB-067: replace `if !isRedirect(resp.StatusCode)
        { return ... }`
// with a bare `return resp, nil` — the 302 flows out as a
        "decode: EOF" error.
```

Result: `TestManagementCookieLogin_ConfigurableAdminPassword` FAILed and the challenge exited with rc=1 — the guard IS load-bearing. Reversed the mutation and re-ran to GREEN.

Honest boundary (§11.4.6)

- The fake is an `httptest.Server` written in Go; it proves the CLIENT speaks the cookie-login protocol. It does NOT prove the client works against every real Jackett version — that is manual QA (§11.4.185) or a live docker-run challenge, tracked separately.
- `scripts/extract-jackett-key.py` reads `ServerConfig.json` from disk and makes NO HTTP calls; cookie-login hardening is not applicable to it — it lives in the Go client that consumes the extracted key.

- Credentials in the fixtures are synthetic (`test-key`, `s3cr3t-dashboard-pw`) — no real Jackett apikey or admin password is ever handled or logged (§11.4.10).

Cross-references

- Companion: `docs/scripts/helixqa-jackett-fake.md`.
- Source of truth (Lava): `lava/lava-api-go/internal/jackett/client.go`.
- Port doc: `docs/PORTING-FROM-LAVA.md` (P4).
- Constitution: §11.4.115 / §11.4.108 / §11.4.146 / §11.4.201 / §11.4.238.